

# Detection is easy, reading order is the tax

A multilingual benchmark of line detection and reading order  
on historical documents

Abhishek Jha

Independent researcher

[huggingface.co/abhishekjha1008](https://huggingface.co/abhishekjha1008) · [github.com/AbhiPandit1](https://github.com/AbhiPandit1)

Working draft, September 2026

## Abstract

On historical documents, finding the text lines on a page is close to solved; deciding the *order* in which to read them is not. On a single column the two questions collapse into one, so the reading-order problem stays hidden until the layout has more than one column. We benchmark five widely used segmentation systems — kraken, a domain-adapted YOLOv8 line detector, Surya, docTR and Tesseract — on 40 pages across eight languages and roughly four centuries, scoring each on two separate quantities: line-detection F at  $\text{IoU} \geq 0.5$ , and a reading-order Kendall  $\tau$  between the predicted order and the ground-truth document order, measured only on the lines that were actually detected. The pure box detectors match the purpose-built segmenter on detection (F 0.91 each) but collapse on reading order for multi-column pages ( $\tau \approx 0.5$ , close to random), while layout-aware systems hold  $\tau \approx 0.97\text{--}1.00$ . The effect is confined to multi-column layouts and is *era-invariant*: it appears identically on a seventeenth-century Swedish handwritten court record and a twentieth-century printed English page. We then show that the gap is not intrinsic to fast detectors: adding a cheap column-clustering ordering stage on top of the same detected boxes raises multi-column  $\tau$  from 0.50 to 0.99 (95% CI [0.98, 1.00]), statistically indistinguishable from the layout-aware segmenter, at roughly nine times the speed. A downstream analysis makes the stake concrete: assuming perfect line recognition, naive ordering alone inflicts a median 82% word error on multi-column pages, which the ordering stage cuts to 2%. We argue that reading order should be reported as a first-class metric alongside recognition accuracy, and release the data, ground truth, predictions and a reproducible harness.

## 1 Introduction

Most published Handwritten Text Recognition (HTR) and OCR results report Character Error Rate (CER) on lines that have already been cut out and placed in the correct reading order, whether produced by end-to-end toolchains such as Transkribus [2] and kraken [1] or by neural line recognisers [3]. This hides a large part of real-world error. When a page has columns, marginalia or tabular structure, the step that decides *in what order* the detected lines are fed to the recogniser determines whether the final transcription is usable at all: a perfect line-level transcription, emitted in the wrong order, still yields scrambled text.

The reason this is easy to overlook is that on a single-column page detection and ordering are the same problem — a top-to-bottom sort of the detected lines is already the correct reading order. The difficulty only becomes visible when the layout branches. This paper isolates that difficulty. We take the same set of historical pages and ask each segmentation system two questions that are usually conflated: **did you find the lines?** and **did you read them in the right order?** We find that modern detectors have largely solved the first and that the second is where systems still diverge sharply — but only on multi-column material.

## 2 Related work

**Layout analysis and line detection.** Historical page segmentation is served by neural systems such as kraken’s b11a [1], Laypa (used in the Loghi pipeline), dhSegment [14] and eynollah, and by baseline-detection methods evaluated in the cBAD competitions [12]. General object detectors such as YOLO [5] have been adapted to line detection with strong results. These systems are typically scored on region or baseline overlap; the order their output implies is rarely evaluated separately.

**Reading-order determination.** Ordering detected regions has a long history, from recursive XY-cut [11] and topological or graph-based sorting to learned approaches: LayoutReader [15] casts reading order as a sequence-to-sequence problem over layout tokens. Most such work targets modern printed documents (forms, scientific papers) where ground-truth reading order is abundant; early-modern manuscript pages with multi-column and marginal structure are far less studied, and rarely with the detector held fixed so that ordering can be isolated.

**HTR pipelines and error attribution.** End-to-end toolchains — Transkribus [2], kraken [1], and neural recognisers such as PyLaia [3] — bundle segmentation, ordering and recognition, which makes it hard to see where error originates. A recurring practical observation is that much of what looks like recognition error is really segmentation or ordering error upstream. This benchmark isolates the ordering component and measures both its intrinsic quality (Kendall  $\tau$ ) and its downstream cost in text error.

## 3 Data

We use 40 scored pages, five per language, across eight languages, drawn from openly licensed sources: the FONDUE collection in the HTR-United catalogue [8], and the public Riksarkivet (Swedish National Archives) HTR data. The set deliberately spans single- and multi-column layouts, printed and handwritten material, and roughly the sixteenth to twentieth centuries (Figure 1). Each page ships with ALTO or PAGE-XML [9] ground truth giving, for every text line, its bounding geometry and its position in document order, which we take as the intended reading sequence. Provenance, era and licence for each language are documented per page in a machine-readable manifest released with the benchmark. No private or restricted archival material is used.

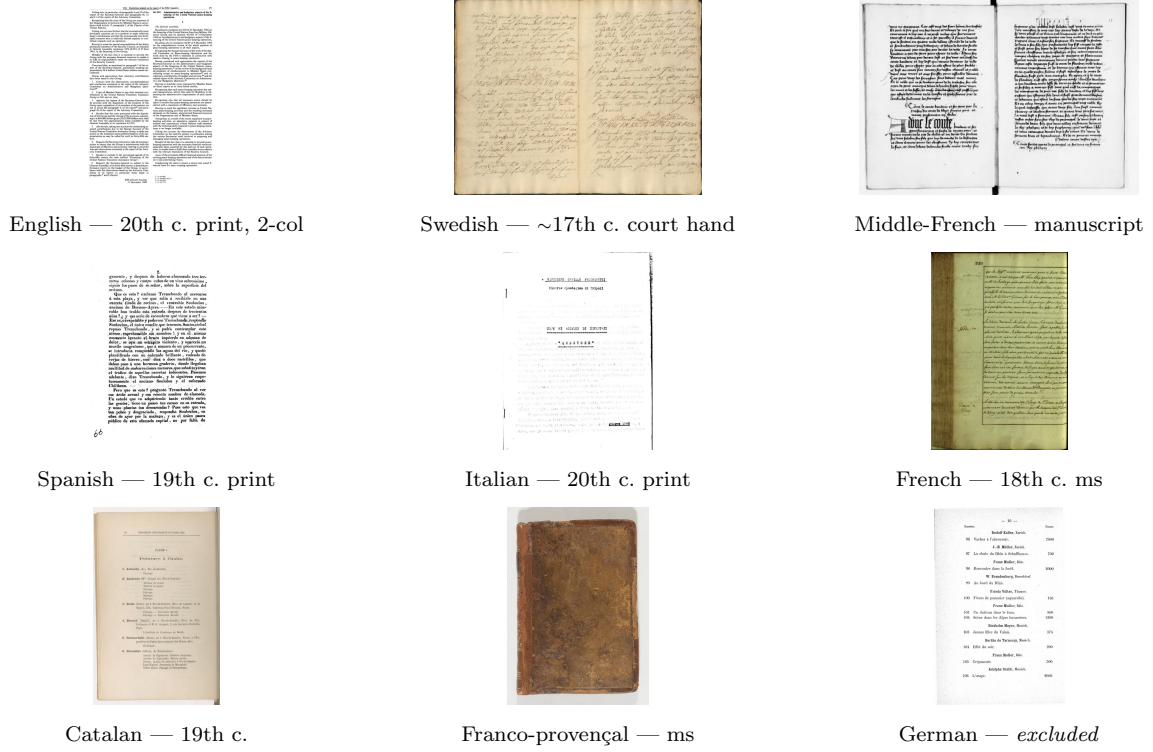
## 4 Systems and method

Every system receives the same page image and must return a list of line boxes *in the order it would read them*; that returned order is what the reading-order metric scores. No per-system tuning is applied; each runs at its documented default. The systems fall into two families.

**Layout-aware systems** emit a reading order themselves. *kraken* [1] (b11a) returns lines already in a region-aware order following baseline geometry, which we keep unchanged. *Tesseract* [4] groups recognised words into lines by its block / paragraph / line indices and keeps its own column-aware layout order. *docTR* [7] (DBNet detector inside `ocr_predictor`) is read in its native block/line traversal order.

**Pure detectors** only localise lines and have no notion of order. The *YOLOv8m* [5] historical line detector and *Surya*’s [6] detection model both return boxes that we sort top-to-bottom by their upper edge — the naive heuristic almost everyone reaches for, and deliberately the weakest ordering, to show what a detector alone provides.

The benchmark is thus a direct test of whether a system’s native reading order beats a geometric top-to-bottom sort, and on which layouts.



**Figure 1:** One page per language. The three multi-column / complex pages (English, Swedish, Middle-French) are where reading order is contested; the single-column pages act as a control. German is retained for completeness but excluded from scoring (Section 9).

## 5 Metrics

Scoring separates the two questions on purpose.

1. **Matching.** Predicted boxes are matched to ground-truth lines greedily at  $\text{IoU} \geq 0.5$ .
2. **Detection F.** Precision/recall on those matches,  $F = 2PR/(P+R)$  — *did you find the lines?*
3. **Reading-order  $\tau$ .** On the matched lines only, Kendall’s  $\tau$  [10] between the predicted order and the ground-truth document order.  $\tau = 1$  is a perfect order;  $\tau \approx 0.5$  means about half the line pairs are swapped, which is what a two-column page does to a top-to-bottom sort — *did you read them in the right order?*
4. **Validity guard.**  $\tau$  is reported for a page only if the system matched  $\geq 60\%$  of ground-truth lines and  $\geq 15$  lines; otherwise there are too few lines for an order to be meaningful and the cell is marked *det-fail*. This prevents a system that found three of fifty lines from claiming a perfect order.
5. **Speed.** Mean wall-clock per page (CPU / Apple MPS).

## 6 Results

Tables 1 and 2 give the picture. Detection is not the differentiator: kraken and the YOLO detector tie at F0.91. Reading order is, and the entire gap lives on the three multi-column languages, where the pure detectors fall to  $\tau \approx 0.5$  while the layout-aware systems stay near 1.0. On single-column pages the naive top-to-bottom sort is already correct, so everyone scores  $\tau \approx 1.0$ : this control isolates the effect.

**Table 1:** Overall, eight scored languages. Detection and reading order split cleanly: the fast detector ties kraken on detection but loses on order.  $\tau$  is averaged over valid pages only.

| System                  | Detection F | Reading-order $\tau$ | Speed (s/pg) | Native order       |
|-------------------------|-------------|----------------------|--------------|--------------------|
| <b>kraken b11a</b>      | 0.91        | <b>0.98</b>          | 10.6         | yes                |
| <b>YOLOv8m</b> detector | 0.91        | 0.77                 | <b>1.2</b>   | no (top-to-bottom) |
| Surya                   | 0.78        | 0.77                 | 5.0          | no                 |
| docTR                   | 0.60        | 0.81                 | 5.3          | native             |
| Tesseract               | 0.59        | 0.98                 | 3.1          | layout-aware       |

**Table 2:** Per-language F /  $\tau$ , grouped by layout. On multi-column pages the detectors collapse to  $\tau \approx 0.5$  despite detecting as well as kraken; on single-column pages the ordering question disappears and every system scores  $\tau \approx 1.0$ . “det-fail” = too few lines detected to score order.

| Language (layout)                       | kraken            | YOLO              | Surya     | Tesseract     | docTR         |
|-----------------------------------------|-------------------|-------------------|-----------|---------------|---------------|
| <i>Multi-column / complex</i>           |                   |                   |           |               |               |
| English (2-col print)                   | 1.00/ <b>0.97</b> | 0.98/ <b>0.50</b> | 0.97/0.50 | 0.97/0.99     | 0.97/0.52     |
| Swedish (court hand)                    | 0.95/ <b>1.00</b> | 0.94/ <b>0.47</b> | 0.81/0.49 | 0.22/det-fail | 0.00/det-fail |
| Middle-French (ms)                      | 0.94/ <b>1.00</b> | 0.94/ <b>0.52</b> | 0.90/0.56 | 0.32/det-fail | 0.62/0.57     |
| <i>Single-column / simple (control)</i> |                   |                   |           |               |               |
| Spanish (print)                         | 0.97/1.00         | 0.93/1.00         | 0.94/1.00 | 0.93/0.95     | 0.91/1.00     |
| Italian (print)                         | 0.81/1.00         | 0.79/1.00         | 0.67/1.00 | 0.81/1.00     | 0.59/1.00     |
| French (ms)                             | 0.99/0.93         | 0.94/0.89         | 0.86/0.95 | 0.53/1.00     | 0.22/0.64     |
| Franco-provençal                        | 0.65/1.00         | 0.78/0.99         | 0.65/1.00 | 0.53/1.00     | 0.64/1.00     |
| Catalan                                 | 0.98/0.98         | 0.96/0.99         | 0.41/1.00 | 0.42/det-fail | 0.86/0.99     |

Figures 2 and 3 show it directly. The detected boxes are almost identical; only the reading path differs. On the English page the detector’s path zig-zags between the two columns on nearly every line; the layout-aware path runs cleanly down one column, then the next.

## 7 Recovering reading order with a cheap ordering stage

The results so far show *that* pure detectors lose reading order, but not *why* in a useful sense: we deliberately gave them the naive top-to-bottom sort. The question a practitioner actually cares about is whether that gap is intrinsic to a fast detector, or an accident of the ordering heuristic. We test this directly by holding detection fixed — the exact same YOLO and Surya boxes — and swapping only the ordering rule.

**Ordering rules.** We compare three geometric rules applied to the detected boxes: (i) *top-to-bottom*, the naive baseline; (ii) *column clustering*, which groups boxes into columns by horizontal gaps between box centres, reads each column top-to-bottom and columns left-to-right; and (iii) *XY-cut* [11], recursive projection profiling. None uses learning or page content; all run in negligible time relative to detection.

**A cheap ordering stage recovers the gap.** Table 3 and Figure 4(a) give the result. Column clustering lifts multi-column  $\tau$  from 0.50 [0.47, 0.54] to 0.99 [0.98, 1.00] — a paired gain of +0.49 [0.44, 0.53] whose confidence interval excludes zero — landing on the layout-aware segmenter’s 0.99 [0.98, 1.00], i.e. statistically indistinguishable from it. XY-cut, by contrast, does *not* help here: full-width running heads bridge the columns and defeat the vertical projection cut, a known failure mode that centre-based clustering avoids. The reading-order gap is therefore not intrinsic to a fast detector; it is an artefact of the ordering heuristic, and a trivial geometric stage removes it.



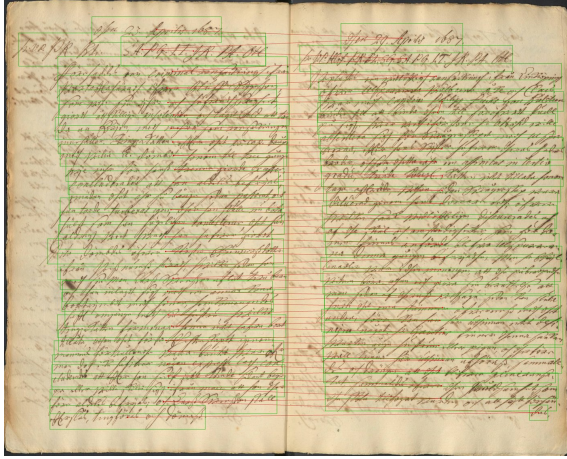
**Figure 2:** Two-column English page. Green: detected lines. Red: reading order handed to the recogniser. Same detection, opposite order.

**Table 3:** Reading-order  $\tau$  on multi-column pages with detection held fixed; only the ordering rule changes. Column clustering closes the gap to the layout-aware segmenter. Means with 95% bootstrap CIs over pages ( $n = 15$ ).

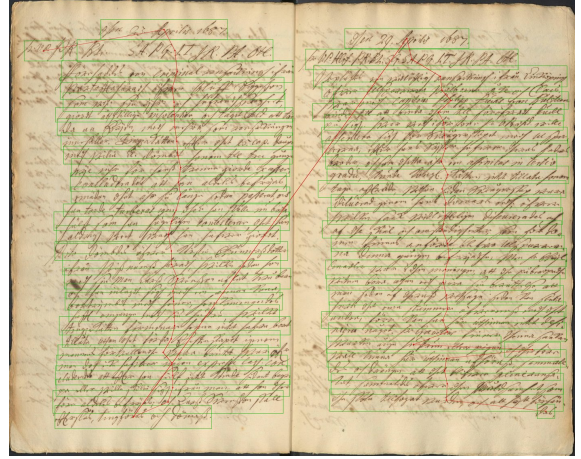
| Ordering on YOLO boxes   | English     | Swedish     | Middle-Fr.  |
|--------------------------|-------------|-------------|-------------|
| top-to-bottom (naive)    | 0.50        | 0.47        | 0.52        |
| XY-cut                   | 0.50        | 0.47        | 0.52        |
| <b>column clustering</b> | <b>0.99</b> | <b>1.00</b> | <b>0.98</b> |
| kraken (naive, ref.)     | 0.97        | 1.00        | 1.00        |

**Downstream stake.** Kendall  $\tau$  is abstract, so we translate it into text error. Assuming a perfect line recogniser (we substitute each detected line with its ground-truth transcription and vary only the order), we concatenate the lines in each system’s order and measure the word error against the correctly ordered text. Figure 4(b): on multi-column pages naive ordering alone inflicts a median 82% word error (Swedish 93%) — a clean line-level transcription rendered unusable purely by order — which column clustering cuts to 2%, matching kraken. On single-column pages every method stays near 2%.

**Scaling and cross-collection check.** To confirm the effect is not an artefact of five pages, we scale the Swedish evaluation to 28 pages drawn from *two* independent Riksarkivet collections — Göta hovrätt appeal-court protocols and the Trolldomskommissionen (witchcraft-commission) records. The result holds and tightens: reading-order  $\tau$  moves from 0.51 [0.47, 0.56] for naive ordering to 1.00 [0.99, 1.00] for column clustering, matching kraken’s 1.00 [1.00, 1.00]; order-induced word error falls from 85% [77, 91] to 1% [0, 2]. That the same recovery appears on a second, independently produced Swedish court-hand collection indicates the finding generalises beyond a single archive.

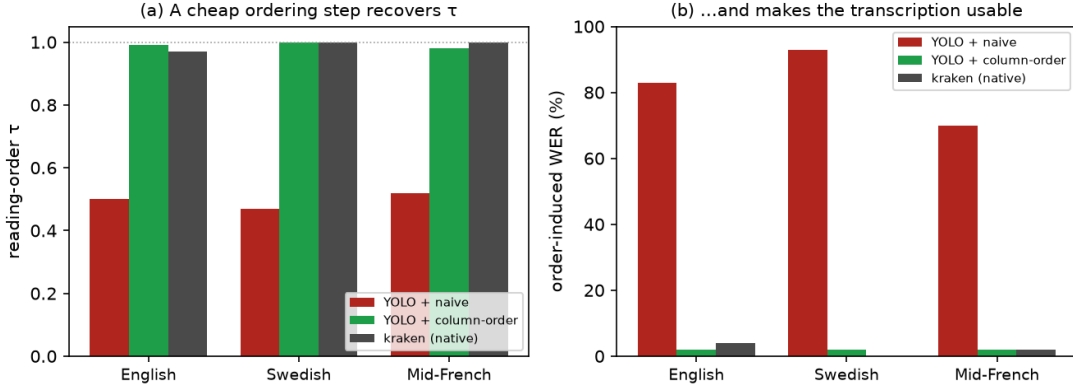


(a) Box detector, top-to-bottom:  $\tau = 0.47$



(b) Layout-aware, native:  $\tau = 1.00$

**Figure 3:** A ~17th-century Swedish handwritten court record (Riksarkivet, Göta hovrätt). Three centuries and one script away from Figure 2, the same tax appears — reading order is a function of layout, not of date or hand.



**Figure 4:** Holding detection fixed on multi-column pages. (a) A column-clustering ordering stage on the fast detector recovers reading-order  $\tau$  to the layout-aware segmenter’s level. (b) The same stage cuts order-induced word error (perfect line recognition assumed) from a median 82% to 2%.

## 8 Discussion

**Fast detection plus cheap ordering is enough.** The headline practical finding is that one does not need a heavier layout-aware segmenter to get correct reading order on these pages. A fast box detector, roughly nine times quicker per page than kraken, plus a column-clustering ordering stage that costs negligible time, matches the layout-aware segmenter on reading-order  $\tau$  (Section 7) and on the resulting text error. The heavy segmenter’s advantage over a naive detector was real but recoverable; it was the ordering step, not the segmentation model, that mattered. The recommendation is therefore not two-tier but simple: pair a fast detector with an explicit ordering stage, and reserve heavier segmenters for cases the geometric rule cannot handle.

**Era-invariance.** That the same detection-versus-order split appears on a seventeenth-century Swedish manuscript and a twentieth-century printed English page — about three centuries and a script apart — indicates the effect is driven by layout rather than by period or hand. This is what makes the finding generalise, and why a benchmark spanning several centuries is a feature rather than noise.

**Recognisers are not detectors.** Tesseract and docTR order well when they detect, but on the Swedish court hand they matched 27% and 0% of lines respectively. Their strong overall order numbers come from the easy printed pages, not the hard manuscripts. Reported as an average this would flatter them; reported per page, with the validity guard, it correctly reads as a detection failure on historical handwriting.

## 9 Limitations and honesty notes

This is a small, deliberately careful pilot, and it is easier to trust because of what it refuses to claim. Reading order is scored only where detection succeeded (the validity guard), so no  $\tau$  is computed on a handful of lines. German is *excluded from all scored tables*: its ground truth is under-annotated — every system, including the layout-aware ones, detects substantially more lines than the ground truth lists — so its F and  $\tau$  are not comparable, and this is a ground-truth coverage issue rather than a system result. Five pages per language is enough to establish the effects cleanly and with confidence intervals that exclude the null, but not to rank systems finely; the ordering stage is evaluated only against document-order ground truth, which is itself an editorial choice on the hardest pages. The natural next steps are (i) scaling to dozens of pages per language for finer ranking; (ii) datasets with explicit region and reading-order ground truth, such as cBAD [12] and DIVA-HisDB [13]; (iii) adding heavier segmenters and pipelines (Transkribus [2], Loghi with Laypa, PaddleOCR PP-Structure) and learned reading-order models; and (iv) reporting additional order metrics beyond Kendall  $\tau$  for robustness. The downstream analysis assumes perfect line recognition to isolate ordering; a full end-to-end study would compose real recognition error with the ordering error.

## 10 Conclusion

On historical documents, line detection is close to solved and is no longer what separates segmentation systems; reading order is, and only on multi-column layouts. But that gap is not intrinsic to fast detectors: a trivial column-clustering ordering stage recovers it to the level of a heavier layout-aware segmenter, statistically and in downstream word error, at a fraction of the cost. The practical takeaway is to pair a fast detector with an explicit ordering stage rather than reach for a heavier model. More broadly, we recommend that reading order be measured and reported as a first-class metric alongside recognition accuracy, and we release the pages, ground truth, predictions and a reproducible scoring harness so the measurement is easy to repeat and extend.

**Data and code.** The benchmark — data, ground truth, predictions and harness — is released openly with a per-page provenance manifest.

## References

- [1] B. Kiessling. *Kraken — an universal text recognizer for the humanities*. Digital Humanities Conference (DH2019), Utrecht, 2019.
- [2] P. Kahle, S. Colutto, G. Hackl, and G. Mühlberger. *Transkribus — a service platform for transcription, recognition and retrieval of historical documents*. In *14th IAPR ICDAR*, pp. 19–24, 2017.
- [3] J. Puigcerver. *Are multidimensional recurrent layers really necessary for handwritten text recognition?* In *14th IAPR ICDAR*, pp. 67–72, 2017.
- [4] R. Smith. *An overview of the Tesseract OCR engine*. In *9th ICDAR*, pp. 629–633, 2007.
- [5] G. Jocher, A. Chaurasia, and J. Qiu. *Ultralytics YOLOv8*. Software, 2023. <https://github.com/ultralytics/ultralytics>.

- [6] V. Paruchuri. *Surya: OCR, layout analysis and reading-order detection*. Software, 2024. <https://github.com/VikParuchuri/surya>.
- [7] Mindee. *docTR: Document Text Recognition*. Software, 2021. <https://github.com/mindee/doctr>.
- [8] A. Chagué, T. Clérice, and L. Romary. *HTR-United: mutualise la donnée d’entraînement en peu de clics*. In *Humanística*, 2021. <https://htr-united.github.io/>.
- [9] S. Pletschacher and A. Antonacopoulos. *The PAGE (Page Analysis and Ground-truth Elements) format framework*. In *20th ICPR*, pp. 257–260, 2010.
- [10] M. G. Kendall. *A new measure of rank correlation*. *Biometrika*, 30(1–2):81–93, 1938.
- [11] G. Nagy and S. Seth. *Hierarchical representation of optically scanned documents*. In *7th ICPR*, pp. 347–349, 1984.
- [12] M. Diem, F. Kleber, S. Fiel, T. Grüning, and B. Gatos. *cBAD: ICDAR2017 competition on baseline detection*. In *14th IAPR ICDAR*, pp. 1355–1360, 2017.
- [13] F. Simistira, M. Seuret, N. Eichenberger, A. Garz, M. Liwicki, and R. Ingold. *DIVA-HisDB: a precisely annotated large dataset of challenging medieval manuscripts*. In *15th ICFHR*, pp. 471–476, 2016.
- [14] S. Ares Oliveira, B. Seguin, and F. Kaplan. *dhSegment: a generic deep-learning approach for document segmentation*. In *16th ICFHR*, pp. 7–12, 2018.
- [15] Z. Wang, Y. Xu, L. Cui, J. Shang, and F. Wei. *LayoutReader: pre-training of text and layout for reading order detection*. In *EMNLP*, pp. 4735–4744, 2021.